

Guía Paso a Paso: Formulario Oracle Forms para Bloqueo/Desbloqueo Manual de Puntos de Venta

Esta guía asume que ya tienes el sistema corriendo (menú principal, login, bibliotecas PL/SQL comunes) y que ya existen las tablas: canal, division, territorio, agencia, punto_venta, cliente, servicio, factura, arreglo_pago, historial_bloqueo_pv y la secuencia seq_historial_bloqueo.

El formulario que vamos a construir se llamará FOR_BLOQUEO_PV.fmb y tendrá 3 funcionalidades:

1. **Listar** puntos de venta filtrando por Canal → División → Territorio → Agencia, con estado ACTIVO/INACTIVO y/o en mora.
 2. **Bloquear / Desbloquear** manualmente un PV seleccionado.
 3. **Desbloquear por arreglo de pago** registrando el acuerdo del cliente.
-

FASE 0: Preparación previa (antes de abrir Forms Builder)

0.1 Crear los objetos de BD que el formulario usará

Necesitamos un paquete PL/SQL que centralice la lógica (así el formulario queda "delgado" y no metemos SQL suelto en triggers). Como SYS o tu esquema dueño:

```
sql
```

```
CREATE OR REPLACE PACKAGE PKG_BLOQUEO_PV AS
```

```
-- Lista PV filtrados por jerarquía y estado
```

```
TYPE t_pv IS RECORD (  
    id_punto_venta NUMBER,  
    nombre_pv      VARCHAR2(150),  
    estado         VARCHAR2(10),  
    direccion      VARCHAR2(200),  
    nombre_agencia VARCHAR2(100),  
    nombre_territorio VARCHAR2(100),
```

```
    nombre_division  VARCHAR2(100),
    nombre_canal     VARCHAR2(100),
    clientes_mora    NUMBER,
    dias_mora_max    NUMBER,
    saldo_mora       NUMBER
```

```
);
```

```
TYPE t_pv_tab IS TABLE OF t_pv;
```

```
FUNCTION fn_listar_pv (
```

```
    p_id_canal      IN NUMBER DEFAULT NULL,
    p_id_division   IN NUMBER DEFAULT NULL,
    p_id_territorio IN NUMBER DEFAULT NULL,
    p_id_agencia    IN NUMBER DEFAULT NULL,
    p_estado        IN VARCHAR2 DEFAULT NULL,
    p_solo_mora     IN VARCHAR2 DEFAULT 'N'
```

```
) RETURN t_pv_tab PIPELINED;
```

```
-- Acciones
```

```
PROCEDURE pr_bloquear_pv (
```

```
    p_id_pv      IN NUMBER,
    p_usuario_app IN VARCHAR2,
    p_motivo      IN VARCHAR2 DEFAULT 'Bloqueo manual'
```

```
);
```

```
PROCEDURE pr_desbloquear_pv (
```

```
    p_id_pv      IN NUMBER,
    p_usuario_app IN VARCHAR2,
    p_motivo      IN VARCHAR2 DEFAULT 'Desbloqueo manual'
```

```
);
```

```
PROCEDURE pr_desbloquear_por_arreglo (
```

```
    p_id_pv      IN NUMBER,
    p_id_cliente  IN NUMBER,
    p_monto_acordado IN NUMBER,
    p_num_cuotas  IN NUMBER,
    p_usuario_app IN VARCHAR2,
    p_obs         IN VARCHAR2 DEFAULT NULL
```

```
);
```

```
END PKG_BLOQUEO_PV;
```

```
/
```

```
CREATE OR REPLACE PACKAGE BODY PKG_BLOQUEO_PV AS
```

```
-- =====
```

```
-- LISTAR PV CON FILTROS
```

```
-- =====
```

```
FUNCTION fn_listar_pv (
```

```
    p_id_canal      IN NUMBER DEFAULT NULL,
```

```
    p_id_division   IN NUMBER DEFAULT NULL,
```

```
    p_id_territorio IN NUMBER DEFAULT NULL,
```

```
    p_id_agencia    IN NUMBER DEFAULT NULL,
```

```
    p_estado        IN VARCHAR2 DEFAULT NULL,
```

```
    p_solo_mora     IN VARCHAR2 DEFAULT 'N'
```

```
) RETURN t_pv_tab PIPELINED
```

```
IS
```

```
BEGIN
```

```
    FOR r IN (
```

```
        SELECT pv.id_punto_venta,
```

```
               pv.nombre_pv,
```

```
               pv.estado,
```

```
               pv.direccion,
```

```
               a.nombre_agencia,
```

```
               t.nombre_territorio,
```

```
               d.nombre_division,
```

```
               ca.nombre_canal,
```

```
               NVL(mora.clientes_mora, 0) AS clientes_mora,
```

```
               NVL(mora.dias_mora_max, 0) AS dias_mora_max,
```

```
               NVL(mora.saldo_mora, 0)   AS saldo_mora
```

```
        FROM punto_venta pv
```

```
        JOIN agencia a ON a.id_agencia = pv.id_agencia
```

```
        JOIN territorio t ON t.id_territorio = a.id_territorio
```

```
        JOIN division d ON d.id_division = t.id_division
```

```
        JOIN canal ca ON ca.id_canal = d.id_canal
```

```
        LEFT JOIN (
```

```
            SELECT c.id_punto_venta,
```

```
                   COUNT(DISTINCT c.id_cliente) AS clientes_mora,
```

```
                   MAX(TRUNC(SYSDATE - f.fecha_vencimiento)) AS dias_mora_max,
```

```
                   SUM(f.saldo_pendiente) AS saldo_mora
```

```
            FROM cliente c
```

```
            JOIN servicio s ON s.id_cliente = c.id_cliente
```

```
            JOIN factura f ON f.id_servicio = s.id_servicio
```

```

        WHERE f.estado IN ('VENCIDA','EN_MORA')
              AND f.saldo_pendiente > 0
              AND f.fecha_vencimiento < TRUNC(SYSDATE)
        GROUP BY c.id_punto_venta
    ) mora ON mora.id_punto_venta = pv.id_punto_venta
    WHERE d.nombre_division = 'Clientes de Mi Negocio'
          AND (p_id_canal IS NULL OR ca.id_canal = p_id_canal)
          AND (p_id_division IS NULL OR d.id_division = p_id_division)
          AND (p_id_territorio IS NULL OR t.id_territorio = p_id_territorio)
          AND (p_id_agencia IS NULL OR a.id_agencia = p_id_agencia)
          AND (p_estado IS NULL OR pv.estado = p_estado)
          AND (p_solo_mora = 'N' OR NVL(mora.clientes_mora,0) > 0)
    ORDER BY pv.nombre_pv
) LOOP
    PIPE ROW (r);
END LOOP;
RETURN;
END fn_listar_pv;

-- =====
-- BLOQUEAR PV
-- =====

PROCEDURE pr_bloquear_pv (
    p_id_pv          IN NUMBER,
    p_usuario_app    IN VARCHAR2,
    p_motivo          IN VARCHAR2 DEFAULT 'Bloqueo manual'
) IS
    v_estado_actual punto_venta.estado%TYPE;
    v_dias_mora      NUMBER;
BEGIN
    SELECT estado INTO v_estado_actual
    FROM punto_venta
    WHERE id_punto_venta = p_id_pv
    FOR UPDATE;

    IF v_estado_actual = 'INACTIVO' THEN
        RAISE_APPLICATION_ERROR(-20010, 'El punto de venta ya está bloqueado.');
```

-- Calcular días de mora máximos del PV

```

BEGIN
```

```

SELECT NVL(MAX(TRUNC(SYSDATE - f.fecha_vencimiento)), 0)
      INTO v_dias_mora
FROM cliente c
JOIN servicio s ON s.id_cliente = c.id_cliente
JOIN factura f ON f.id_servicio = s.id_servicio
WHERE c.id_punto_venta = p_id_pv
      AND f.estado IN ('VENCIDA', 'EN_MORA')
      AND f.saldo_pendiente > 0;
EXCEPTION WHEN NO_DATA_FOUND THEN v_dias_mora := 0;
END;

UPDATE punto_venta
      SET estado = 'INACTIVO'
      WHERE id_punto_venta = p_id_pv;

INSERT INTO historial_bloqueo_pv (
      id_historial, id_punto_venta, estado_anterior, estado_nuevo,
      motivo, usuario_bd, usuario_app, origen, dias_mora
) VALUES (
      seq_historial_bloqueo.NEXTVAL,
      p_id_pv, 'ACTIVO', 'INACTIVO',
      p_motivo, USER, p_usuario_app, 'MANUAL', v_dias_mora
);

COMMIT;
EXCEPTION
      WHEN OTHERS THEN
            ROLLBACK;
            RAISE;
END pr_bloquear_pv;

-- =====
-- DESBLOQUEAR PV
-- =====

PROCEDURE pr_desbloquear_pv (
      p_id_pv          IN NUMBER,
      p_usuario_app    IN VARCHAR2,
      p_motivo          IN VARCHAR2 DEFAULT 'Desbloqueo manual'
) IS
      v_estado_actual punto_venta.estado%TYPE;
BEGIN

```

```

SELECT estado INTO v_estado_actual
  FROM punto_venta
 WHERE id_punto_venta = p_id_pv
    FOR UPDATE;

IF v_estado_actual = 'ACTIVO' THEN
  RAISE_APPLICATION_ERROR(-20011, 'El punto de venta ya está activo.');
```

END IF;

```

UPDATE punto_venta
  SET estado = 'ACTIVO'
 WHERE id_punto_venta = p_id_pv;

INSERT INTO historial_bloqueo_pv (
  id_historial, id_punto_venta, estado_anterior, estado_nuevo,
  motivo, usuario_bd, usuario_app, origen
) VALUES (
  seq_historial_bloqueo.NEXTVAL,
  p_id_pv, 'INACTIVO', 'ACTIVO',
  p_motivo, USER, p_usuario_app, 'MANUAL'
);

COMMIT;

EXCEPTION
  WHEN OTHERS THEN ROLLBACK; RAISE;

END pr_desbloquear_pv;

-- =====
-- DESBLOQUEAR POR ARREGLO DE PAGO
-- =====

PROCEDURE pr_desbloquear_por_arreglo (
  p_id_pv          IN NUMBER,
  p_id_cliente     IN NUMBER,
  p_monto_acordado IN NUMBER,
  p_num_cuotas     IN NUMBER,
  p_usuario_app    IN VARCHAR2,
  p_obs            IN VARCHAR2 DEFAULT NULL
) IS
  v_id_arreglo NUMBER;
BEGIN
  -- Crear el arreglo de pago
```

```

v_id_arreglo := seq_arreglo_pago.NEXTVAL;

INSERT INTO arreglo_pago (
    id_arreglo, id_cliente, fecha_acuerdo,
    monto_acordado, num_cuotas, estado
) VALUES (
    v_id_arreglo, p_id_cliente, SYSDATE,
    p_monto_acordado, p_num_cuotas, 'ACTIVO'
);

-- Desbloquear el PV
UPDATE punto_venta
    SET estado = 'ACTIVO'
    WHERE id_punto_venta = p_id_pv
    AND estado = 'INACTIVO';

-- Registrar en historial
INSERT INTO historial_bloqueo_pv (
    id_historial, id_punto_venta, estado_anterior, estado_nuevo,
    motivo, usuario_bd, usuario_app, origen, id_cliente, observaciones
) VALUES (
    seq_historial_bloqueo.NEXTVAL,
    p_id_pv, 'INACTIVO', 'ACTIVO',
    'Desbloqueo por arreglo de pago ID: ' || v_id_arreglo,
    USER, p_usuario_app, 'MANUAL',
    p_id_cliente, p_obs
);

COMMIT;
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END pr_desbloquear_por_arreglo;

END PKG_BLOQUEO_PV;
/

```

0.2 Otorgar permisos

sql

```
GRANT EXECUTE ON PKG_BLOQUEO_PV TO TU_USUARIO_FORMS;
```

GRANT SELECT, INSERT, UPDATE ON cliente, servicio, factura, arreglo_pago, punto_venta, historial_bloqueo_pv TO TU_USUARIO_FORMS;
GRANT SELECT ON canal, division, territorio, agencia TO TU_USUARIO_FORMS;

FASE 1: Crear el módulo de formulario en Forms Builder

1.1 Abrir Forms Builder y crear módulo

1. Abre **Oracle Forms Builder**.
2. **File** → **New** → **Form** (o **Ctrl+N** → **Form**).
3. Se abre el **Object Navigator**. Verás un nodo raíz llamado **MODULE1**.
4. Guarda inmediatamente: **File** → **Save As...** → nombre **FOR_BLOQUEO_PV.fmb** en tu carpeta de trabajo.

1.2 Configurar las propiedades del módulo

Selecciona el nodo del módulo (**FOR_BLOQUEO_PV**) en el Object Navigator y en el Property Palette ajusta:

Propiedad	Valor
Name	FOR_BLOQUEO_PV
First Navigation Data Block	BLK_FILTRO
Console Window	WIN_MAIN (o el que uses)
Menu Module	MENU_PRINCIPAL (el menú existente)

1.3 Crear las ventanas (Windows) y canvas

En el Object Navigator, expande **Windows**:

1. Clic derecho sobre **Windows** → **Create** → renombra a **WIN_MAIN**.
2. Clic derecho sobre **Canvas** → **Create** → tipo **Content** → renombra a **CVS_FILTRO**.
3. Clic derecho sobre **Canvas** → **Create** → tipo **Content** → renombra a **CVS_PV**.

4.Clic derecho sobre **Canvases** → **Create** → tipo **Content** → renombra a **CVS_ACCIONES**.

Asigna:

- WIN_MAIN como Window de todos los canvas (Property Palette → Window).
- Todos los canvas visibles al inicio (Visible = Yes).

FASE 2: Construir los bloques de datos

Diseño general:

text

[CANAL ▼]	[DIVISIÓN ▼]	[TERRITORIO ▼]	[AGENCIA ▼]		← BLK_FILTRO	
[Estado: ()Todos ()Activos ()Inactivos] [☑ Solo mora]						
[🔍 Buscar]						
ID	PV	Agencia	Estado	Mora	Saldo	← BLK_PV
...						
Observaciones: [_____]						
[🚫 Bloquear] [✅ Desbloquear] [📄 Arreglo Pago]						← BLK_ACCION

2.1 Bloque BLK_FILTRO (Control Block, no basado en tabla)

- 1.En **Data Blocks** → **Create** → selecciona **Build a new data block manually** → **Next**.
- 2.Nombre del bloque: **BLK_FILTRO**. **No** marques "Database Data Block". → **Next** → **Finish**.
- 3.Ahora crea los ítems dentro del bloque. Clic derecho sobre **BLK_FILTRO** → **Items** → **Create** varias veces:

Item	Type	Prompt	Data Type	Width
IT_CANAL	List Item	Canal	Number	30
IT_DIVISION	List Item	División	Number	30
IT_TERRITORIO	List Item	Territorio	Number	30
IT_AGENCIA	List Item	Agencia	Number	30

Item	Type	Prompt	Data Type	Width
IT_ESTADO	Radio Group	Estado	Char	-
IT_SOLO_MORA	Check Box	Solo mora	Char	-
BT_BUSCAR	Push Button	Buscar	-	-

Detalle importante: Los List Items (IT_CANAL, IT_DIVISION, etc.) los vamos a llenar dinámicamente en tiempo de ejecución, así que NO les pongas elementos fijos. Sí debes configurar:

- IT_CANAL: Data Type = Number, List Style = Poplist, **Mapping of Other Values = 0.**
- Igual para los otros 3.

2.2 Bloque BLK_PV (Data Block basado en punto_venta)

1.**Data Blocks** → **Create** → **Next.**

2.Nombre: BLK_PV.

3.Selecciona **Yes** para "Based on a table" y elige PUNTO_VENTA → **Next.**

4.Selecciona las columnas: id_punto_venta, nombre_pv, estado, direccion. → **Next.**

5.Desmarca la opción de incluir todos los ítems por defecto → **Finish.**

Ahora **elimina** las columnas de base de tabla que no necesitas. Deja solo:

- ID_PUNTO_VENTA (Primary Key, Query Only = No, Update Allowed = No)
- NOMBRE_PV (Query Only = Yes, Insert/Update Allowed = No)
- ESTADO (Query Only = Yes, Insert/Update Allowed = No)
- DIRECCION (Query Only = Yes)

Agrega ítems NO-basados-en-tabla (Clic derecho sobre BLK_PV → Items → Create) y en el Property Palette cambia **Database Item = No:**

Item	Type	Data Type	Función
IT_NOMBRE_CANAL	Text Item	Char	Mostrar canal
IT_NOMBRE_DIVISION	Text Item	Char	Mostrar división
IT_NOMBRE_TERRITORIO	Text Item	Char	Mostrar territorio
IT_NOMBRE_AGENCIA	Text Item	Char	Mostrar agencia

Item	Type	Data Type	Función
IT_CLIENTES_MORA	Text Item	Number	Cuántos clientes en mora
IT_DIAS_MORA	Text Item	Number	Días máximos de mora
IT_SALDO_MORA	Text Item	Number	Saldo total en mora
IT_ID_CLIENTE_MORA	Text Item	Number	ID del cliente con más mora

Estos se llenarán con un `POST-QUERY` trigger (ver Fase 4).

En el **Property Palette del bloque** `BLK_PV`:

•**Query Data Source Type** = `FROM clause query` (¡clave! porque vamos a usar una query personalizada con joins).

•**Query Data Source Name**:

sql

```
(SELECT pv.id_punto_venta,
        pv.nombre_pv,
        pv.estado,
        pv.direccion
FROM punto_venta pv
JOIN agencia a ON a.id_agencia = pv.id_agencia
JOIN territorio t ON t.id_territorio = a.id_territorio
JOIN division d ON d.id_division = t.id_division
JOIN canal ca ON ca.id_canal = d.id_canal
WHERE d.nombre_division = 'Clientes de Mi Negocio')
```

•**WHERE Clause** (para que Forms lo use al hacer `EXECUTE_QUERY`):

Déjala **vacía**. Vamos a filtrar programáticamente con `SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, ...)` desde el botón Buscar.

2.3 Bloque `BLK_ACCION` (Control Block)

1.**Data Blocks** → **Create** → **Build manually**.

2.Nombre: `BLK_ACCION`.

3.Crea los ítems:

Item	Type	Prompt	Notas
IT_OBSERVACIONES	Text Item	Observaciones	Multi-line (Height=60, Multi-line=Yes)
IT_MONTO_ARR	Text Item	Monto arreglo	Number
IT_CUOTAS_ARR	Text Item	N° cuotas	Number, Range 1–36
BT_BLOQUEAR	Push Button	Bloquear	-
BT_DESBLOQUEAR	Push Button	Desbloquear	-
BT_ARREGLO	Push Button	Arreglo de Pago	-
BT_SALIR	Push Button	Salir	-

FASE 3: Layout (arrastrar ítems a los canvas)

1. Abre el **Layout Editor** para CVS_FILTRO.
2. Arrastra los ítems de BLK_FILTRO desde el Object Navigator. Distribúyelos en una fila horizontal.
3. Abre el Layout Editor para CVS_PV.
4. Arrastra los ítems de BLK_PV. Los ítems "no database" van a la derecha o debajo.
5. Abre el Layout Editor para CVS_ACCIONES.
6. Arrastra los botones y el campo de observaciones.

Sugerencia de **Prompt** y **Width**:

Item	Prompt	Width
IT_CANAL	Canal:	25
IT_DIVISION	División:	25
IT_TERRITORIO	Territorio:	25
IT_AGENCIA	Agencia:	25
IT_ESTADO	Estado:	—
IT_SOLO_MORA	Solo mora	—

FASE 4: Lógica — Triggers y Program Units

4.1 Poblar los List Items al iniciar el formulario

Crea un **Program Unit** de tipo Procedure llamado PR_CARGAR_LISTAS:

```
sql

PROCEDURE PR_CARGAR_LISTAS IS
BEGIN
    -- Canal
    CLEAR_LIST('BLK_FILTRO.IT_CANAL');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_CANAL', 0, '(Todos)', 0);
    FOR r IN (SELECT id_canal, nombre_canal FROM canal ORDER BY nombre_canal) LOOP
        ADD_LIST_ELEMENT('BLK_FILTRO.IT_CANAL', r.id_canal, r.nombre_canal, r.id_canal);
    END LOOP;

    -- División: solo 'Clientes de Mi Negocio'
    CLEAR_LIST('BLK_FILTRO.IT_DIVISION');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', 0, '(Todas)', 0);
    FOR r IN (SELECT id_division, nombre_division
              FROM division
              WHERE nombre_division = 'Clientes de Mi Negocio') LOOP
        ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', r.id_division, r.nombre_division,
r.id_division);
    END LOOP;

    -- Territorio (se filtra por división)
    CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);

    -- Agencia (se filtra por territorio)
    CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
    ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);

    -- Valores por defecto
    :BLK_FILTRO.IT_CANAL      := 0;
    :BLK_FILTRO.IT_DIVISION   := 0;
    :BLK_FILTRO.IT_TERRITORIO := 0;
```

```

:BLK_FILTRO.IT_AGENCIA      := 0;
:BLK_FILTRO.IT_ESTADO      := 'T';  -- T = Todos
:BLK_FILTRO.IT_SOLO_MORA   := 'N';

```

```
END;
```

Crea un **Form Trigger** WHEN-NEW-FORM-INSTANCE:

```
sql
```

```
BEGIN
```

```
-- Guardar el usuario de la app para auditoría
```

```
:GLOBAL.usuario_app := GET_APPLICATION_PROPERTY(USERNAME);
```

```
-- Cargar Listas
```

```
PR_CARGAR_LISTAS;
```

```
-- Poner el foco en el filtro
```

```
GO_ITEM('BLK_FILTRO.IT_CANAL');
```

```
END;
```

4.2 Cascada: al cambiar Canal, recargar Divisiones

Trigger WHEN-LIST-CHANGED en BLK_FILTRO.IT_CANAL:

```
sql
```

```
DECLARE
```

```
v_id_canal NUMBER := :BLK_FILTRO.IT_CANAL;
```

```
BEGIN
```

```
CLEAR_LIST('BLK_FILTRO.IT_DIVISION');
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', 0, '(Todas)', 0);
```

```
FOR r IN (SELECT id_division, nombre_division
```

```
FROM division
```

```
WHERE nombre_division = 'Clientes de Mi Negocio'
```

```
AND (v_id_canal = 0 OR id_canal = v_id_canal)) LOOP
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_DIVISION', r.id_division, r.nombre_division,
r.id_division);
```

```
END LOOP;
```

```
:BLK_FILTRO.IT_DIVISION := 0;
```

```
-- Limpiar las listas hijas
```

```

CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);
:BLK_FILTRO.IT_TERRITORIO := 0;
CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
:BLK_FILTRO.IT_AGENCIA := 0;

```

END;

Trigger WHEN-LIST-CHANGED en BLK_FILTRO.IT_DIVISION:

sql

DECLARE

```
v_id_div NUMBER := :BLK_FILTRO.IT_DIVISION;
```

BEGIN

```
CLEAR_LIST('BLK_FILTRO.IT_TERRITORIO');
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', 0, '(Todos)', 0);
```

```
FOR r IN (SELECT id_territorio, nombre_territorio
```

```
FROM territorio
```

```
WHERE (v_id_div = 0 OR id_division = v_id_div)) LOOP
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_TERRITORIO', r.id_territorio, r.nombre_territorio,
r.id_territorio);
```

```
END LOOP;
```

```
:BLK_FILTRO.IT_TERRITORIO := 0;
```

```
CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
```

```
:BLK_FILTRO.IT_AGENCIA := 0;
```

END;

Trigger WHEN-LIST-CHANGED en BLK_FILTRO.IT_TERRITORIO:

sql

DECLARE

```
v_id_terr NUMBER := :BLK_FILTRO.IT_TERRITORIO;
```

BEGIN

```
CLEAR_LIST('BLK_FILTRO.IT_AGENCIA');
```

```
ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', 0, '(Todas)', 0);
```

```
FOR r IN (SELECT id_agencia, nombre_agencia
```

```
FROM agencia
```

```

        WHERE (v_id_terr = 0 OR id_territorio = v_id_terr)) LOOP
            ADD_LIST_ELEMENT('BLK_FILTRO.IT_AGENCIA', r.id_agencia, r.nombre_agencia,
r.id_agencia);
        END LOOP;

        :BLK_FILTRO.IT_AGENCIA := 0;
END;

```

4.3 Botón BUSCAR — construir el WHERE dinámico

Trigger WHEN-BUTTON-PRESSED en BLK_FILTRO.BT_BUSCAR:

sql

```

DECLARE
    v_where    VARCHAR2(4000);
    v_estado   VARCHAR2(1) := :BLK_FILTRO.IT_ESTADO;
BEGIN
    v_where := 'd.nombre_division = ''Clientes de Mi Negocio''';

    IF :BLK_FILTRO.IT_CANAL > 0 THEN
        v_where := v_where || ' AND ca.id_canal = ' || :BLK_FILTRO.IT_CANAL;
    END IF;

    IF :BLK_FILTRO.IT_DIVISION > 0 THEN
        v_where := v_where || ' AND d.id_division = ' || :BLK_FILTRO.IT_DIVISION;
    END IF;

    IF :BLK_FILTRO.IT_TERRITORIO > 0 THEN
        v_where := v_where || ' AND t.id_territorio = ' || :BLK_FILTRO.IT_TERRITORIO;
    END IF;

    IF :BLK_FILTRO.IT_AGENCIA > 0 THEN
        v_where := v_where || ' AND a.id_agencia = ' || :BLK_FILTRO.IT_AGENCIA;
    END IF;

    IF v_estado = 'A' THEN
        v_where := v_where || ' AND pv.estado = ''ACTIVO''';
    ELSIF v_estado = 'I' THEN
        v_where := v_where || ' AND pv.estado = ''INACTIVO''';
    END IF;

    IF :BLK_FILTRO.IT_SOLO_MORA = 'Y' THEN

```

```

v_where := v_where || ' AND EXISTS (SELECT 1 FROM cliente c ' ||
                    ' JOIN servicio s ON s.id_cliente = c.id_cliente ' ||
                    ' JOIN factura f ON f.id_servicio = s.id_servicio ' ||
                    ' WHERE c.id_punto_venta = pv.id_punto_venta ' ||
                    ' AND f.estado IN (''VENCIDA'', ''EN_MORA'') ' ||
                    ' AND f.saldo_pendiente > 0)';

END IF;

SET_BLOCK_PROPERTY('BLK_PV', DEFAULT_WHERE, v_where);
GO_BLOCK('BLK_PV');
EXECUTE_QUERY;

END;

```

4.4 POST-QUERY — traer datos extra de cada PV

Trigger POST-QUERY en BLK_PV:

```

sql

DECLARE
    v_cliente    NUMBER;
    v_dias       NUMBER;
    v_saldo      NUMBER;
    v_cantidad   NUMBER;

BEGIN
    -- Datos de la jerarquía
    SELECT ca.nombre_canal, d.nombre_division, t.nombre_territorio, a.nombre_agencia
        INTO :BLK_PV.IT_NOMBRE_CANAL,
             :BLK_PV.IT_NOMBRE_DIVISION,
             :BLK_PV.IT_NOMBRE_TERRITORIO,
             :BLK_PV.IT_NOMBRE_AGENCIA
    FROM punto_venta pv
    JOIN agencia a ON a.id_agencia = pv.id_agencia
    JOIN territorio t ON t.id_territorio = a.id_territorio
    JOIN division d ON d.id_division = t.id_division
    JOIN canal ca ON ca.id_canal = d.id_canal
    WHERE pv.id_punto_venta = :BLK_PV.ID_PUNTO_VENTA;

    -- Datos de mora
    BEGIN

```

```

SELECT COUNT(DISTINCT c.id_cliente),
       MAX(TRUNC(SYSDATE - f.fecha_vencimiento)),
       SUM(f.saldo_pendiente),
       MAX(c.id_cliente) KEEP (DENSE_RANK FIRST ORDER BY TRUNC(SYSDATE -
f.fecha_vencimiento) DESC)
  INTO v_cantidad, v_dias, v_saldo, v_cliente
  FROM cliente c
  JOIN servicio s ON s.id_cliente = c.id_cliente
  JOIN factura f ON f.id_servicio = s.id_servicio
 WHERE c.id_punto_venta = :BLK_PV.ID_PUNTO_VENTA
       AND f.estado IN ('VENCIDA', 'EN_MORA')
       AND f.saldo_pendiente > 0;

:BLK_PV.IT_CLIENTES_MORA := NVL(v_cantidad, 0);
:BLK_PV.IT_DIAS_MORA     := NVL(v_dias, 0);
:BLK_PV.IT_SALDO_MORA    := NVL(v_saldo, 0);
:BLK_PV.IT_ID_CLIENTE_MORA := v_cliente;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    :BLK_PV.IT_CLIENTES_MORA := 0;
    :BLK_PV.IT_DIAS_MORA     := 0;
    :BLK_PV.IT_SALDO_MORA    := 0;
    :BLK_PV.IT_ID_CLIENTE_MORA := NULL;

END;
END;

```

4.5 Botón BLOQUEAR

Trigger WHEN-BUTTON-PRESSED en BLK_ACCION.BT_BLOQUEAR:

```

sql

DECLARE
  v_id_pv      NUMBER := :BLK_PV.ID_PUNTO_VENTA;
  v_estado     VARCHAR2(10) := :BLK_PV.ESTADO;
  v_usuario    VARCHAR2(50) := NVL(:GLOBAL.usuario_app, USER);
  v_obs        VARCHAR2(500) := :BLK_ACCION.IT_OBSERVACIONES;
  v_resp       NUMBER;

BEGIN
  IF v_id_pv IS NULL THEN

```

```

        MESSAGE('Debe seleccionar un punto de venta.');
```

```

        RAISE FORM_TRIGGER_FAILURE;
```

```

    END IF;
```

```

    IF v_estado = 'INACTIVO' THEN
        MESSAGE('El punto de venta ya está bloqueado.');
```

```

        RAISE FORM_TRIGGER_FAILURE;
```

```

    END IF;
```

```

    v_resp := SHOW_ALERT('ALT_CONFIRMA'); -- Alerta que definiremos abajo
```

```

    IF v_resp != ALERT_BUTTON1 THEN
        RETURN;
```

```

    END IF;
```

```

    PKG_BLOQUEO_PV.PR_BLOQUEAR_PV(
        p_id_pv      => v_id_pv,
        p_usuario_app => v_usuario,
        p_motivo      => NVL(v_obs, 'Bloqueo manual desde formulario')
    );
```

```

    MESSAGE('Punto de venta bloqueado correctamente.');
```

```

    :BLK_PV.ESTADO := 'INACTIVO';
```

```

    :BLK_ACCION.IT_OBSERVACIONES := NULL;
```

```

    -- Refrescar el listado para reflejar el cambio
```

```

    GO_BLOCK('BLK_PV');
```

```

    EXECUTE_QUERY;
```

```

EXCEPTION
```

```

    WHEN OTHERS THEN
        MESSAGE('Error al bloquear: ' || SQLERRM);
```

```

        RAISE FORM_TRIGGER_FAILURE;
```

```

END;
```

4.6 Botón DESBLOQUEAR

Trigger WHEN-BUTTON-PRESSED en BLK_ACCION.BT_DESBLOQUEAR:

```

sql
```

```

DECLARE
```

```
v_id_pv      NUMBER := :BLK_PV.ID_PUNTO_VENTA;
v_estado     VARCHAR2(10) := :BLK_PV.ESTADO;
v_usuario    VARCHAR2(50) := NVL(:GLOBAL.usuario_app, USER);
v_obs        VARCHAR2(500) := :BLK_ACCION.IT_OBSERVACIONES;
v_resp       NUMBER;
```

```
BEGIN
```

```
IF v_id_pv IS NULL THEN
    MESSAGE('Debe seleccionar un punto de venta.');
```

```
    RAISE FORM_TRIGGER_FAILURE;
END IF;
```

```
IF v_estado = 'ACTIVO' THEN
    MESSAGE('El punto de venta ya está activo.');
```

```
    RAISE FORM_TRIGGER_FAILURE;
END IF;
```

```
v_resp := SHOW_ALERT('ALT_CONFIRMA');
```

```
IF v_resp != ALERT_BUTTON1 THEN
    RETURN;
END IF;
```

```
PKG_BLOQUEO_PV.PR_DESBLOQUEAR_PV(
    p_id_pv      => v_id_pv,
    p_usuario_app => v_usuario,
    p_motivo     => NVL(v_obs, 'Desbloqueo manual desde formulario')
);
```

```
MESSAGE('Punto de venta desbloqueado correctamente.');
```

```
:BLK_PV.ESTADO := 'ACTIVO';
:BLK_ACCION.IT_OBSERVACIONES := NULL;
```

```
GO_BLOCK('BLK_PV');
EXECUTE_QUERY;
```

```
EXCEPTION
```

```
WHEN OTHERS THEN
    MESSAGE('Error al desbloquear: ' || SQLERRM);
    RAISE FORM_TRIGGER_FAILURE;
```

```
END;
```

4.7 Botón ARREGLO DE PAGO

Trigger WHEN-BUTTON-PRESSED en BLK_ACCION.BT_ARREGLO:

sql

DECLARE

```
v_id_pv      NUMBER := :BLK_PV.ID_PUNTO_VENTA;
v_id_cliente NUMBER := :BLK_PV.IT_ID_CLIENTE_MORA;
v_monto      NUMBER := :BLK_ACCION.IT_MONTO_ARR;
v_cuotas     NUMBER := :BLK_ACCION.IT_CUOTAS_ARR;
v_usuario    VARCHAR2(50) := NVL(:GLOBAL.usuario_app, USER);
v_obs        VARCHAR2(500) := :BLK_ACCION.IT_OBSERVACIONES;
v_resp       NUMBER;
```

BEGIN

```
IF v_id_pv IS NULL THEN
    MESSAGE('Debe seleccionar un punto de venta.');
```

RAISE FORM_TRIGGER_FAILURE;

```
END IF;
```



```
IF v_id_cliente IS NULL THEN
    MESSAGE('El PV seleccionado no tiene clientes en mora.');
```

RAISE FORM_TRIGGER_FAILURE;

```
END IF;
```



```
IF v_monto IS NULL OR v_monto <= 0 THEN
    MESSAGE('Ingrese un monto válido para el arreglo de pago.');
```

RAISE FORM_TRIGGER_FAILURE;

```
END IF;
```



```
IF v_cuotas IS NULL OR v_cuotas < 1 OR v_cuotas > 36 THEN
    MESSAGE('El número de cuotas debe estar entre 1 y 36.');
```

RAISE FORM_TRIGGER_FAILURE;

```
END IF;
```



```
v_resp := SHOW_ALERT('ALT_CONFIRMA');
```

```
IF v_resp != ALERT_BUTTON1 THEN
    RETURN;
```

```
END IF;
```

```
PKG_BLOQUEO_PV.PR_DESBLOQUEAR_POR_ARREGLO(
```

```

        p_id_pv          => v_id_pv,
        p_id_cliente     => v_id_cliente,
        p_monto_acordado => v_monto,
        p_num_cuotas     => v_cuotas,
        p_usuario_app    => v_usuario,
        p_obs            => v_obs
    );

    MESSAGE('Arreglo de pago registrado y PV desbloqueado.');
```

```

:BLK_PV.ESTADO := 'ACTIVO';
:BLK_ACCION.IT_MONTO_ARR := NULL;
:BLK_ACCION.IT_CUOTAS_ARR := NULL;
:BLK_ACCION.IT_OBSERVACIONES := NULL;

GO_BLOCK('BLK_PV');
EXECUTE_QUERY;

EXCEPTION
    WHEN OTHERS THEN
        MESSAGE('Error al registrar arreglo: ' || SQLERRM);
        RAISE FORM_TRIGGER_FAILURE;

END;
```

4.8 Botón SALIR

```

sql

BEGIN
    EXIT_FORM(NO_VALIDATE);
END;
```

4.9 Crear la alerta ALT_CONFIRMAR

En el Object Navigator: **Alerts** → **Create** → nombrarla **ALT_CONFIRMAR**. En el Property Palette:

- Title: Confirmación
- Message: ¿Está seguro de realizar esta acción?
- Button1 Label: Sí
- Button2 Label: No
- Default Alert Button: Button1
- Style: Stop

FASE 5: Integrar el formulario al menú principal

Tienes 2 formas:

Opción A: Menú Oracle Forms tradicional (.mmb / .mmx)

1. Abre tu módulo de menú `MENU_PRINCIPAL.mmb` en Forms Builder.
2. Selecciona el ítem padre donde quieras colgarlo, por ejemplo **"Cobranza"** (créalo si no existe).
3. Agrega un ítem hijo:

- **Label:** Bloqueo de Puntos de Venta
- **Command Type:** PL/SQL
- **Command:** `CALL_FORM('FOR_BLOQUEO_PV');`

Opción B: Botón en un menú de formulario

Si tu menú es un formulario (con botones), crea un botón "Bloqueo de PV" con:

```
sql
BEGIN
    CALL_FORM('FOR_BLOQUEO_PV', HIDE, NO_REPLACE, NO_QUERY_ONLY);
END;
```

Parámetros de `CALL_FORM`:

- `HIDE` → oculta el formulario padre mientras se usa este.
- `NO_REPLACE` → puede volver al menú al cerrar.
- `NO_QUERY_ONLY` → permite DML.

Registrar en `formspath`

Asegúrate de que la carpeta donde está `FOR_BLOQUEO_PV.fmx` esté en el `FORMS_PATH` del servidor o del cliente. En `default.env` (o `formsweb.cfg`):

text

FORMS_PATH=C:\mis_formularios;C:\otra_ruta

FASE 6: Compilar y probar

6.1 Compilar

1. **File** → **Save** (Ctrl+S) → guarda el .fmb.
2. **Program** → **Compile Module** (Ctrl+K).
3. Verifica que no haya errores en la ventana **Compiler Output**.

6.2 Compilar también el menú (si aplica)

En MENU_PRINCIPAL.mmb → **Program** → **Compile** → genera .mmx.

6.3 Ejecutar

1. **Program** → **Run Form** (Ctrl+R).
2. Se abrirá el formulario. Verifica:
 - ☒ Los List Items se llenan con datos.
 - ☒ El efecto cascada (Canal → División → Territorio → Agencia) funciona.
 - ☒ Al presionar Buscar, lista los PV correctos.
 - ☒ Al seleccionar un PV con mora, aparecen los campos IT_CLIENTES_MORA, IT_DIAS_MORA, IT_SALDO_MORA.
 - ☒ Al presionar Bloquear, cambia el estado y se registra en historial.
 - ☒ Al presionar Desbloquear, vuelve a ACTIVO.
 - ☒ Al presionar Arreglo de Pago, crea el registro en arreglo_pago y desbloquea.

6.4 Validar en base de datos

Desde PL/SQL Developer, verifica el historial:

sql

```
SELECT * FROM historial_bloqueo_pv ORDER BY fecha_accion DESC;  
SELECT * FROM arreglo_pago ORDER BY fecha_acuerdo DESC;  
SELECT id_punto_venta, nombre_pv, estado FROM punto_venta;
```

FASE 7: Detalles que suelen olvidarse

7.1 Manejo del usuario logueado

Si tu sistema tiene login, cambia en `WHEN-NEW-FORM-INSTANCE`:

sql

```
:GLOBAL.usuario_app := NVL(:GLOBAL.usuario_logueado, GET_APPLICATION_PROPERTY(USERNAME));
```

Y en el `POST-LOGON` de tu formulario de login asigna

```
:GLOBAL.usuario_logueado := :usuario.
```

7.2 Bloquear edición directa de columnas

En el Property Palette de `BLK_PV.ID_PUNTO_VENTA`:

- `Update Allowed` = No
- `Insert Allowed` = No

En `BLK_PV.NOMBRE_PV`, `BLK_PV.ESTADO`, `BLK_PV.DIRECCION`:

- `Query Only` = Yes
- `Update Allowed` = No
- `Insert Allowed` = No

Así el usuario no puede modificar nada desde la grilla, solo verla.

7.3 Poner el formulario en "modo consulta" al abrir

En `WHEN-NEW-FORM-INSTANCE`, agrega:

```
sql

SET_BLOCK_PROPERTY('BLK_PV', INSERT_ALLOWED, PROPERTY_FALSE);
SET_BLOCK_PROPERTY('BLK_PV', UPDATE_ALLOWED, PROPERTY_FALSE);
```

7.4 Refrescar tras cada acción

Ya lo hacemos con `EXECUTE_QUERY` después de cada acción. Alternativa más rápida (si el filtro no cambió): `:BLK_PV.ESTADO := 'INACTIVO'` y `SYNCHRONIZE`.

7.5 Permisos

Asegúrate de que el usuario de BD con el que corre Forms tenga:

```
sql

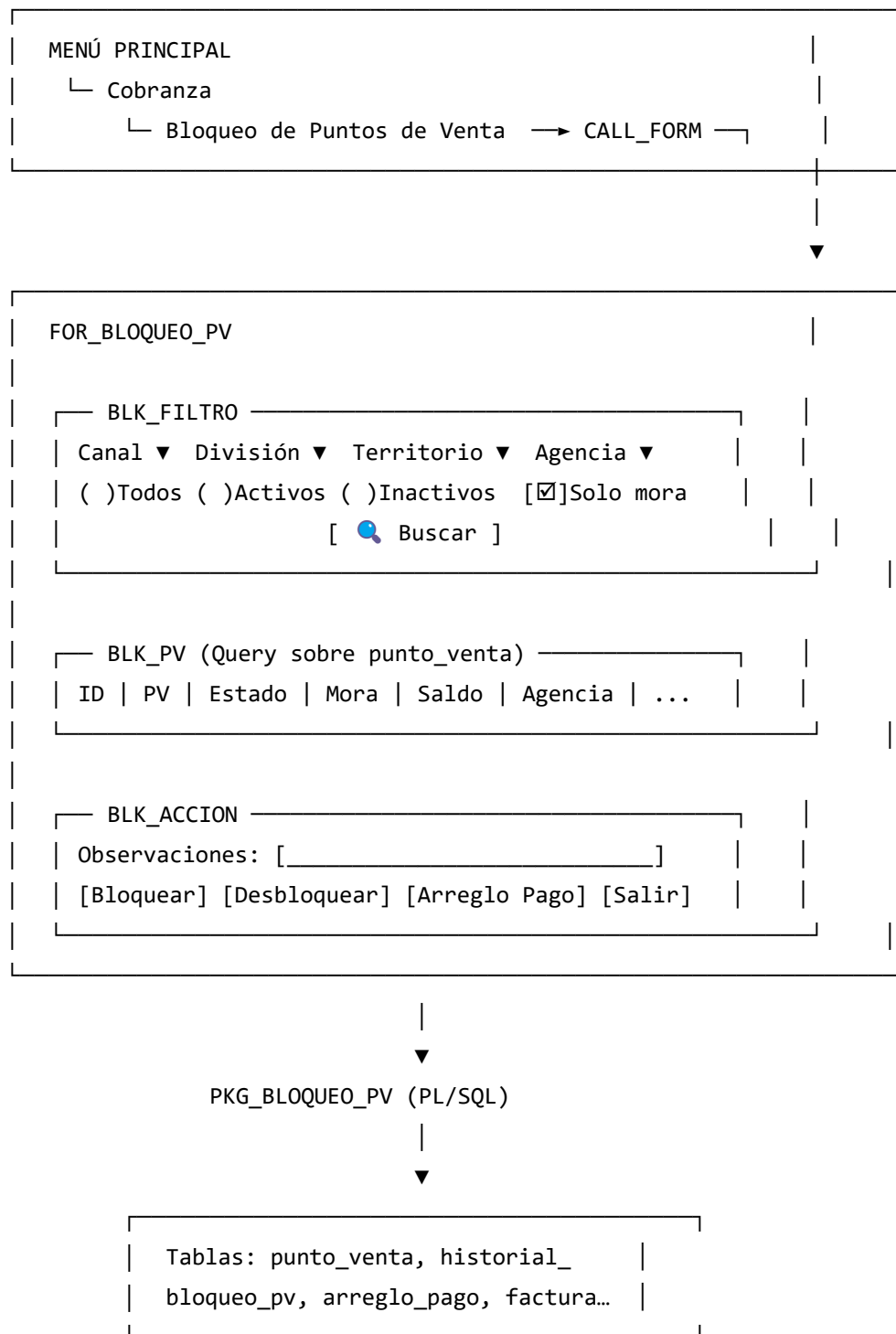
GRANT EXECUTE ON PKG_BLOQUEO_PV TO TU_USUARIO_FORMS;
GRANT SELECT, INSERT, UPDATE ON punto_venta, historial_bloqueo_pv, arreglo_pago TO
TU_USUARIO_FORMS;
GRANT SELECT ON canal, division, territorio, agencia, cliente, servicio, factura TO
TU_USUARIO_FORMS;
```

7.6 Iconos y layout bonito

En la Property Palette de cada botón, cambia `Iconic` a `Yes` y asigna un `Icon Name` (necesitas tener un `.ico` en la carpeta `FORMS_PATH`). Si no tienes íconos, deja los botones como texto.

Resumen visual de la arquitectura

text



Checklist final antes de entregar

- ☐

Scripts SQL ejecutados (tablas, secuencia, paquete `PKG_BLOQUEO_PV`).

- ☐

Permisos concedidos al usuario de Forms.

- ☐

Módulo `FOR_BLOQUEO_PV.fmb` creado y compilado sin errores.

- ☐

Menú modificado y compilado (`.mmx` regenerado).

- ☐

`FORMS_PATH` incluye la carpeta del nuevo `.fmx`.

- ☐

Probado: listar, filtrar, bloquear, desbloquear, arreglo de pago.

- ☐

Verificado en BD: `historial_bloqueo_pv` y `arreglo_pago` registran correctamente.

- ☐

Validado que un usuario sin permiso no puede modificar el PV directamente.